

Hands-On-Learning: Deploy and Monitor an LLM Application for HealthTech Inc

Learning Objectives

By the end of this activity, learners will be able to:

- Containerize and deploy an LLM inference service with secure-by-default configurations.
- Implement layered defenses (identity-aware networking, input validation) to mitigate LLM-specific threats.
- Instrument, monitor, and stress test a production deployment to meet latency and uptime SLAs.



Description

In this hands-on practice, you are HealthTech Inc.'s lead DevOps engineer, tasked with deploying and hardening a medical document analysis LLM that serves thousands of clinicians. Your operational artifacts will feed the enterprise rollout plan. You will deliver a Kubernetes-based deployment with layered security, autoscaling, monitoring dashboards, and incident runbooks that satisfy reliability and compliance targets.

Structure of the Activity

Dataset Information/Resources Required

The lab uses synthetic payloads that simulate clinician document submissions. Security policies and sample prompts are provided in the starter repository under requirements.

Dataset:

→ readme file (**m3_deploy_monitor_llm_application_dataset_readme.md**)

Tools Used:

- Docker 24+
- kubectl, kind/minikube
- Linux Desktop Image (Lab environment)
- Helm
- Prometheus Operator, Grafana
- k6 or Locust, trivy, dockle

Setup Requirements

→ Access to starter code file: `m3_lab_starter.ipynb` in the Linux desktop lab environment

→ Docker daemon running

→ Install required CLIs: `pip install -U kubernetes python-kubernetes rich click`

→ Starter notebook provides shell command wrappers, manifests, and monitoring configuration scaffolds

Scenario

HealthTech Inc.'s compliance review requires hardened containers and an auditable cluster setup for their medical document analysis LLM serving thousands of clinicians. As the lead DevOps engineer, you must demonstrate secure, observable deployment before load testing. Operations leadership needs proof that the system sustains surge traffic while remaining compliant with healthcare regulations. You must deliver production-ready infrastructure that balances security, performance, and reliability.

Objective

Deliver a Kubernetes-based deployment with layered security, autoscaling, monitoring dashboards, and incident runbooks that satisfy reliability and compliance targets.



Instructions for Participants

Activity 1: Harden Containers and Build the Cluster

Objective

HealthTech's compliance review requires hardened containers and an auditable cluster setup.



Steps to Complete

Step 1: Build the LLM API Docker image using the provided multi-stage Dockerfile with a non-root user.

Brief guidance: Use the following commands:

```
!docker build -t healthtech/llm-api:secure -f docker/Dockerfile . !trivy  
image --exit-code 1 healthtech/llm-api:secure
```

Step 2: Run trivy and dockle scans; remediate critical findings.

Brief guidance: Capture scan reports and attach them to the compliance package.

Step 3: Provision a Kubernetes healthtech-prod namespace with RBAC roles for DevOps, operators, and auditors.

Brief guidance: Use the following commands:

```
!kubectl create namespace healthtech-prod !kubectl apply -f k8s/rbac/devops-  
role.yaml -n healthtech-prod
```


Try It Yourself — Container Hardening Checklist:

< YOUR CODE HERE >: add commands to enforce read-only root filesystem and drop Linux capabilities

Expected Outcome: Scans exit with zero critical vulnerabilities; RBAC manifests apply without warnings.

Tips:

- Which vulnerabilities surfaced during scanning, and how did you remediate them?
- How do namespace-scoped roles limit blast radius for misconfigurations?
- Enable image signing with Cosign and document verification steps for audits.
- Execute `tests/test_rbac.py` to ensure roles and role bindings exist for each persona.
- Expected output: RBAC coverage: PASS when roles are detected.

Activity 2: Deploy the LLM Service with Layered Defenses

Objective

You must demonstrate secure, observable deployment before load testing.



Steps to Complete

Step 1: Apply the deployment manifest with liveness/readiness probes, resource requests/limits, and PodDisruptionBudgets.

Brief guidance: Use the following commands:

```
!kubectl apply -f k8s/deployment.yaml -n healthtech-prod !kubectl apply -f  
k8s/service.yaml -n healthtech-prod !kubectl apply -f k8s/ingress.yaml -n  
healthtech-prod
```

Step 2: Configure Istio or NGINX Ingress with mutual TLS, JWT validation, and rate limiting at the gateway.

Brief guidance: Ensure ingress only allows authenticated requests.

Step 3: Implement prompt filtering hooks and response sanitizers using the provided middleware template.

Brief guidance: Use the following code pattern:

```
from security import deploy_prompt_filters  
deploy_prompt_filters(namespace='healthtech-prod', model_name='llm-api')
```

Try It Yourself — Defense-in-Depth Tweaks:

< YOUR CODE HERE >: add ingress annotations for rate limiting and JWT issuer validation

Expected Outcome: Liveness probes pass, and ingress shows authenticated requests only.

Tips:

- How do you verify that mutual TLS is enforced between services?
- Which abuse monitoring metrics catch prompt injection attempts early?
- Inject synthetic malicious prompts and confirm filters block them with audit logs.
- Log denied prompts to a secure bucket for threat hunting.
- Run `tests/test_security_controls.py` to verify gateway and middleware configurations.
- Expected output: Security controls: PASS when checks succeed.

Activity 3: Monitor, Stress Test, and Document Runbooks

Objective

Operations leadership needs proof that the system can sustain surge traffic while remaining compliant.



Steps to Complete

Step 1: Install Prometheus Operator and Grafana via Helm; deploy prebuilt dashboards from dashboards/.

Brief guidance: Use the following commands:

```
!helm upgrade --install monitoring stack/prometheus-operator -n healthtech-prod  
!helm upgrade --install grafana grafana/grafana -n healthtech-prod
```

Step 2: Run k6 or Locust load tests (100 → 10,000 VUs) and observe autoscaling behavior.

Brief guidance: Use the following commands:

```
!k6 run load_tests/healthtech_k6.js
```

Step 3: Execute chaos tests (pod kill, database outage) and update the incident runbook with remediation steps.

Brief guidance: Document notification channels (PagerDuty, Slack) within the runbook for rapid response. Use the following code pattern:

```
from chaos import kill_pod kill_pod(namespace='healthtech-prod',  
label_selector='app=llm-api')
```

Try It Yourself — Alert Rule Builder:

```
alert_rules = [ # < YOUR CODE HERE >: add alert definitions for latency, error  
rate, and pod restarts ]
```

Expected Outcome: Autoscaler keeps p95 latency under 500 ms; alerts fire when thresholds are breached.

Tips:

- Which metrics triggered scale-out events during load tests?
- How will you detect and respond to accuracy drift in production?
- Export Grafana dashboards and share them with compliance auditors as evidence.
- Execute `tests/test_alert_rules.py` to confirm alert definitions deploy correctly.
- Expected output: Alert rules: PASS when all required alerts exist.

Compile Your Findings

1. Requirement 1: Container Hardening and Cluster Setup

Deliverable: Docker image hardened and vulnerability scans pass without critical findings.

Include:

- Multi-stage Dockerfile with non-root user
- Trivy and dockle scan reports (zero critical vulnerabilities)
- Kubernetes healthtech-prod namespace with RBAC roles

- Container hardening checklist (read-only filesystem, dropped capabilities)
- Image signing verification steps (optional)

2. Requirement 2: Secure Deployment with Layered Defenses

Deliverable: Kubernetes deployment, ingress, and middleware enforce layered security controls.

Include:

- Deployment manifest with liveness/readiness probes and resource limits
- PodDisruptionBudgets configuration
- Ingress configuration with mutual TLS, JWT validation, and rate limiting

- Prompt filtering middleware and response sanitizers
- Security controls verification (PASS status)

3. Requirement 3: Monitoring, Load Testing, and Runbooks

Deliverable: Monitoring stack operational with dashboards, alerts, and autoscaling metrics. Chaos and load tests documented with outcomes and remediation steps.

Include:

- Prometheus Operator and Grafana deployment
- Prebuilt dashboards for latency, error rates, and security signals

- 
- Load test results (100 → 10,000 VUs) with autoscaling behavior
 - Chaos test outcomes (pod kill, database outage)
 - Alert rules (latency, error rate, pod restarts)
 - Incident runbook with remediation steps and notification channels
 - P95 latency under 500 ms with autoscaling active
 - Program runs without errors

Summary

This activity provides hands-on experience with the practical application of containerization, security hardening, and observability practices for production-ready LLM deployment. Participants have developed competency in building secure infrastructure, implementing defense-in-depth strategies, and validating system reliability under stress, which are directly applicable to enterprise production deployments.

Key takeaways:

- Secure deployment requires layered defenses spanning container, cluster, and gateway
- Building observability stacks that capture latency, error rates, and security signals in real time
- Continuous testing (load + chaos) validates SLAs and safeguards patient trust

Common Issues & Solutions

Problem: Trivy scan fails due to network restrictions.

Solution: Use the offline vulnerability database packaged in ./assets/trivy-dump and rerun with --db-repository flag.

Problem: Autoscaler fails to scale beyond 5 pods.

Solution: Increase resource limits, review HPA target metrics, and confirm cluster node pool capacity.

Resources

Primary Materials:

→ **m3_lab_starter.ipynb** - Starter notebook with shell command wrappers and configuration scaffolds

→ **m3_deploy_monitor_llm_application_dataset_readme.md** - Overview of sample payloads and security policies

Additional Resources:

- Anthropic's Production Security: Lessons from Claude's Deployment (reading)
- HIPAA compliance checklist
- Module 3 videos: LLM Deployment Strategies and Architectures, Security and Privacy Considerations for LLM Deployment, Monitoring and Performance Optimization

Solution/Exemplar:

- **m3_lab_solution.ipynb** - Fully commented reference implementation

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.